SEGUNDO PARCIAL · PROYECTO CLIENTE/SERVIDOR CON BASE DE DATOS

# PetCare

Sistema web para la gestión de una clínica veterinaria

---

*Documento técnico del proyecto*

**Materia:** Base de Datos

**Docente:** Edwin Torrado


**Integrantes**

Camilo Ernesto Amaya López (0192265)

Miguel Felipe Quintero Santos (0192334)

Ángel Jhosep Sánchez Oviedo (0192411)


**Video:** https://youtu.be/KULm3711eSk

Noviembre de 2026

# Contenido

**Resumen** ..... **3**
**1. Descripción del problema** ..... **3**
Problemas que resuelve ..... 3
Por qué necesita una base de datos ..... 3
Ciclo de atención ..... 3
Actores ..... 4
**2. Cumplimiento de las características generales** ..... **4**
**3. Diseño y modelo de la base de datos** ..... **5**
Tablas y razón de existir ..... 5
Relaciones ..... 6
Integridad ..... 6
Tipos de datos ..... 7
Normalización ..... 7
Diagrama del modelo relacional ..... 8
**4. Arquitectura del sistema** ..... **9**
Las tres capas ..... 10
Recorrido de una operación: registrar un tutor ..... 11
Conexión y seguridad ..... 11
**5. Operaciones CRUD** ..... **11**
Ejemplos del código ..... 12
**6. Módulos funcionales** ..... **12**
Sitio público y solicitudes de cita ..... 13
Tablero ..... 13
Tutores ..... 13
Mascotas ..... 13
Citas y solicitudes web ..... 14
Atención clínica ..... 14
Vacunación ..... 15
Facturación y pagos ..... 15
Personal y usuarios ..... 15
Catálogos ..... 15
Reportes ..... 16
**7. Informes** ..... **16**
**8. Entregables y estructura del comprimido** ..... **17**
Estructura del archivo comprimido ..... 18
Generar el esquema y el backup ..... 18
**9. Video explicativo en YouTube** ..... **18**
**10. Instrucciones para ejecutar el proyecto** ..... **19**
1. Crear la base de datos ..... 19
2. Configurar la aplicación ..... 19
3. Instalar y ejecutar ..... 19
4. Usuarios de prueba ..... 20
**Anexo. Modelo relacional completo** ..... **21**
Seguridad y personas ..... 21
Mascota ..... 21
Atención clínica ..... 21
Prescripción y vacunación ..... 22
Facturación ..... 22
Sitio público ..... 23

# Resumen

PetCare es un sistema web cliente/servidor para administrar una clínica veterinaria. Funciona sobre una base de datos PostgreSQL 16 de 29 tablas relacionadas, con 39 claves foráneas, y todas sus pantallas leen y escriben datos reales.

# 1. Descripción del problema

Una clínica veterinaria lleva su agenda, historias clínicas, vacunas, medicamentos y cobros en cuadernos, hojas de cálculo y mensajes sueltos. PetCare reúne toda esa información en una sola base de datos que comparten la recepción, los veterinarios y la administración. El caso es simulado y se basa en el funcionamiento habitual de una clínica.

## Problemas que resuelve

- **Citas cruzadas.** Sin control, un veterinario puede quedar con dos turnos a la misma hora.
- **Historial disperso.** Los diagnósticos escritos a mano no permiten saber qué enfermedades son más frecuentes ni revisar la evolución de una mascota.
- **Vacunas olvidadas.** Nadie sabe cuándo le toca el refuerzo a cada mascota.
- **Inventario sin control.** Se vende un medicamento que ya no está en existencia.
- **Cobros confusos.** No se sabe cuánto se ha pagado de cada factura ni cuánto falta.
- **Pedidos por internet sin orden.** Las solicitudes de cita llegan por varios canales y se pierden.

## Por qué necesita una base de datos

La información de la clínica está encadenada: un tutor tiene mascotas, una mascota tiene citas, una cita produce una consulta, y de la consulta salen diagnósticos, tratamientos, recetas y una factura. Varias personas la consultan y modifican al mismo tiempo, y debe conservarse por años porque el historial clínico es un registro permanente. Además, hay reglas que nunca deben romperse, como no cobrar más del total o no dejar el stock en negativo. Una base de datos relacional guarda todo eso de forma persistente, evita datos duplicados y hace cumplir las reglas aunque falle la aplicación.

En este dominio el paciente no es el cliente: el paciente es la mascota y la relación comercial es con su tutor. Esa separación (`mascota` y `cliente`) es la primera decisión del modelo.

## Ciclo de atención

1. El tutor pide una cita desde el sitio público, sin crear cuenta.
2. La recepcionista revisa la solicitud y la convierte en una cita con un veterinario.
3. El veterinario atiende la cita y registra la consulta con signos vitales y peso.
4. En la consulta registra diagnósticos, tratamientos, una receta y, si aplica, vacunas.

5. La recepción emite la factura al tutor y registra los pagos, totales o parciales.
6. La administración revisa los informes de ingresos, servicios y diagnósticos.

### Actores

| Actor | Qué hace en el sistema |
|---|---|
| Administrador | Usa todo el sistema: catálogos, personal, usuarios e informes |
| Veterinario | Atiende sus citas y registra consultas, diagnósticos, tratamientos, recetas y vacunas. Lee todo el historial, pero solo modifica lo que él firmó |
| Recepcionista | Maneja la agenda, las solicitudes web, los tutores, las mascotas, la facturación y los pagos. No ve la parte clínica |
| Visitante (tutor) | Consulta servicios y equipo en el sitio público y envía solicitudes de cita sin registrarse |

**Fuera del alcance:** hospitalización, kardex de inventario, cuentas para los tutores y pagos en línea.

## 2. Cumplimiento de las características generales

PetCare cumple los cuatro requisitos mínimos del parcial. Las cifras de la tabla se contaron directamente en la base `petcare` el 4 de noviembre de 2026.

| Requisito del parcial | Cómo lo cumple PetCare | Dónde se explica |
|---|---|---|
| Arquitectura cliente/servidor | El navegador (cliente) se comunica con un servidor Next.js, que a su vez se conecta al servidor de base de datos PostgreSQL 16 | Sección 4 |
| Mínimo 12 tablas | 29 tablas relacionadas, cada una con una razón de existir | Sección 3 y anexo |
| Clave primaria en cada tabla | 29 claves primarias, una por tabla | Anexo |
| Relaciones con claves foráneas | 39 claves foráneas, cada una con su política de borrado | Sección 3 |
| Restricciones de integridad | 56 restricciones `CHECK`, 27 `UNIQUE`, 3 dominios, 1 restricción `EXCLUDE` y 10 triggers | Sección 3 |
| CRUD completo | `INSERT`, `SELECT`, `UPDATE` y `DELETE` desde los módulos de la aplicación | Secciones 5 y 6 |
| Informes | Módulo de reportes y tablero de indicadores, calculados con 10 vistas | Sección 7 |
| Datos reales | Toda la información sale de PostgreSQL. Nada está escrito en el código | Sección 5 |
| Video en YouTube | Publicado; el enlace está en la sección 9 | Sección 9 |

La base trae datos de demostración cargados con `07_demo.sql`: 7 tutores, 10 mascotas, 4 veterinarios, 11 citas, 4 consultas, 9 vacunas aplicadas, 3 facturas y 2 pagos, entre otros.

# 3. Diseño y modelo de la base de datos

La base `petcare` tiene 29 tablas en PostgreSQL 16, escritas a mano en SQL y sin migraciones de un ORM. Se agrupan en cinco bloques más la tabla de solicitudes del sitio público, y ninguna existe solo para completar el mínimo de 12.

## Tablas y razón de existir

| Tabla | Bloque | Razón de existir |
|---|---|---|
| `rol` | Seguridad y personas | Perfiles de acceso: Administrador, Veterinario y Recepcionista |
| `usuario` | Seguridad y personas | Credencial para entrar al sistema. Guarda solo el hash de la contraseña (scrypt), nunca el texto plano |
| `veterinario` | Seguridad y personas | Profesional que atiende. Firma las citas, consultas y vacunas |
| `especialidad` | Seguridad y personas | Catálogo de especialidades: medicina general, cirugía, dermatología, entre otras |
| `veterinario_especialidad` | Seguridad y personas | Intermedia N:M. Un veterinario tiene varias especialidades y una es la principal |
| `cliente` | Seguridad y personas | Tutor responsable de la mascota y titular de las facturas |
| `especie` | Mascota | Catálogo: canino, felino, ave, roedor, conejo y reptil |
| `raza` | Mascota | Catálogo de razas, cada una de una especie |
| `mascota` | Mascota | El paciente y núcleo del sistema |
| `condicion_medica` | Mascota | Catálogo de alergias y condiciones crónicas o congénitas |
| `mascota_condicion` | Mascota | Intermedia N:M. Guarda la severidad que la condición tiene en esa mascota |
| `registro_peso` | Mascota | Historial de pesos. El peso actual es el registro más reciente |
| `cita` | Atención clínica | Turno de una mascota con un veterinario en una fecha y hora |
| `consulta` | Atención clínica | Atención realizada, con anamnesis, signos vitales y examen físico |
| `diagnostico` | Atención clínica | Catálogo normalizado de diagnósticos, con código y nombre |
| `consulta_diagnostico` | Atención clínica | Intermedia N:M. Indica si el diagnóstico es presuntivo, definitivo o descartado, y cuál es el principal |
| `tratamiento` | Atención clínica | Plan terapéutico de una consulta, con fechas y estado |
| `receta` | Prescripción y vacunación | Receta de una consulta. La base le asigna el folio |
| `medicamento` | Prescripción y vacunación | Catálogo con precio, stock y stock mínimo |
| `receta_detalle` | Prescripción y vacunación | Intermedia N:M. Dosis, frecuencia, duración y vía de cada medicamento recetado |
| `vacuna` | Prescripción y vacunación | Catálogo con intervalo de refuerzo y edad mínima |
| `vacuna_especie` | Prescripción y vacunación | Intermedia N:M. Especies a las que se puede aplicar cada vacuna |

| Tabla | Bloque | Razón de existir |
|---|---|---|
| `aplicacion_vacuna` | Prescripción y vacunación | Vacunas aplicadas a cada mascota, con lote y fecha del próximo refuerzo |
| `servicio` | Facturación | Catálogo de servicios con precio base. También se muestra en el sitio público |
| `factura` | Facturación | Documento de cobro al tutor. El total y el estado se calculan solos |
| `factura_detalle` | Facturación | Líneas de la factura. Cada línea es un servicio o un medicamento, nunca los dos |
| `metodo_pago` | Facturación | Catálogo: efectivo, tarjetas, transferencia y Nequi / Daviplata |
| `pago` | Facturación | Pagos recibidos. Permite pagos parciales y varios medios en una factura |
| `solicitud_cita` | Sitio público | Pedidos de cita de visitantes. Esperan ahí hasta que la recepción los convierte en una cita real |

## Relaciones

| Tipo | Cómo se implementa | Casos |
|---|---|---|
| 1:1 opcional | Clave foránea con `UNIQUE` | `usuario`–`veterinario`, `cita`–`consulta`, `consulta`–`receta`, `cita`–`solicitud_cita` |
| 1:N | Clave foránea en la tabla del lado «muchos» | `cliente`–`mascota`, `mascota`–`cita`, `veterinario`–`consulta`, `factura`–`pago`, entre otras |
| N:M | Tabla intermedia con clave primaria compuesta | `veterinario_especialidad`, `mascota_condicion`, `consulta_diagnostico`, `receta_detalle`, `vacuna_especie` |
| N:M con historia | Tabla intermedia con clave propia, porque el mismo par se repite en el tiempo | `aplicacion_vacuna` (mascota–vacuna, para registrar refuerzos) y `factura_detalle` (factura–servicio o medicamento) |

Las tablas intermedias guardan los datos que pertenecen al vínculo y no a sus extremos. Por ejemplo, la dosis no es propiedad del medicamento ni de la receta, sino de recetar ese medicamento en esa receta.

## Integridad

La base hace cumplir sus reglas por sí sola, aunque los datos lleguen por `psql` y no por la aplicación.

- **Claves foráneas (39).** Cada una tiene una política de borrado. `RESTRICT` en 25: el historial clínico y los catálogos en uso no se borran. `CASCADE` en 7: solo donde la fila hija es una parte del padre, como las líneas de una factura. `SET NULL` en 7: referencias de trazabilidad, como la consulta de la que salió un peso.
- **Clave foránea compuesta.** `mascota (raza_id, especie_id)` apunta a `raza (id, especie_id)`. Así es imposible registrar un gato de raza labrador.

- **Dominios (3).** `dom_dinero` (`NUMERIC(12,2)`, nunca negativo), `dom_email` y `dom_telefono` validan el formato una sola vez para todas las columnas que los usan.
- **Restricciones `CHECK` (56).** Estados válidos, temperatura entre 30 y 45 °C, peso mayor que 0, fecha de nacimiento no futura, cantidades positivas y stock no negativo, entre otras.
- **`UNIQUE` (27) e índices únicos parciales.** Documento del tutor, matrícula del veterinario, número de factura, un solo diagnóstico principal por consulta y un solo peso por mascota y día.
- **`EXCLUDE` (1).** Un veterinario no puede tener dos citas que se crucen. Las canceladas liberan el horario.
- **Triggers (10).** Calculan el fin de la cita, el número de factura y el folio de receta; recalculan totales y estado de la factura; descuentan el stock al facturar; impiden cobrar más del total o modificar una factura cerrada; exigen que la consulta sea de la misma mascota y veterinario que su cita; validan la especie de la vacuna y calculan su próximo refuerzo.

## Tipos de datos

- Claves primarias `INTEGER GENERATED ALWAYS AS IDENTITY`, y `SMALLINT` en catálogos pequeños.
- Dinero en `NUMERIC(12,2)`, nunca `FLOAT`, porque los decimales binarios descuadran los totales.
- Teléfono y microchip en `VARCHAR`, porque llevan símbolos y ceros a la izquierda.
- `DATE` para hechos de un día (nacimiento, vacuna, peso) y `TIMESTAMPTZ` para momentos exactos (cita, consulta, pago).
- Estados en `VARCHAR` con `CHECK`, que se pueden ampliar sin recrear un tipo `ENUM`.

## Normalización

El modelo está en tercera forma normal. Los datos que se pueden calcular no se guardan dos veces:

- La cita no guarda el tutor; se obtiene por `mascota → cliente`.
- La receta no guarda la mascota ni el veterinario; se obtienen por `receta → consulta`.
- La edad, el peso actual, el estado de vacunación y el saldo de una factura se calculan en vistas.
- El diagnóstico es un catálogo y no texto libre, para que «Gastritis» y «gastritis» no sean dos enfermedades distintas.

Hay cuatro datos derivados que sí se guardan a propósito, y cada uno lo protege un trigger o una columna generada: los totales de la factura y el precio de cada línea (una factura no puede cambiar si mañana sube un precio), el stock del medicamento y el fin de la cita (la restricción `EXCLUDE` necesita un rango de tiempo).

## Diagrama del modelo relacional

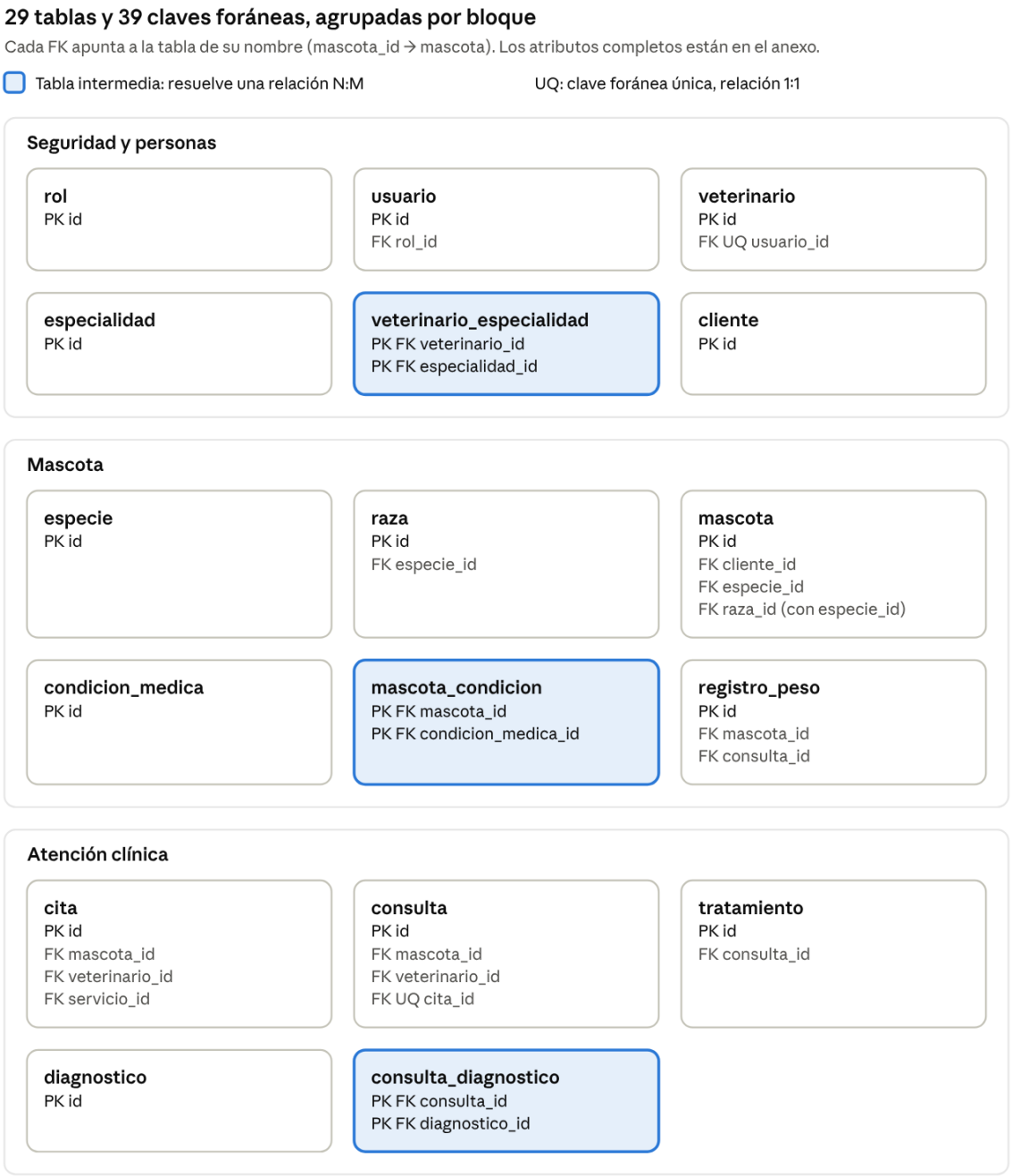



*Figura 1. Modelo relacional (1 de 2): seguridad y personas, mascota y atención clínica.*

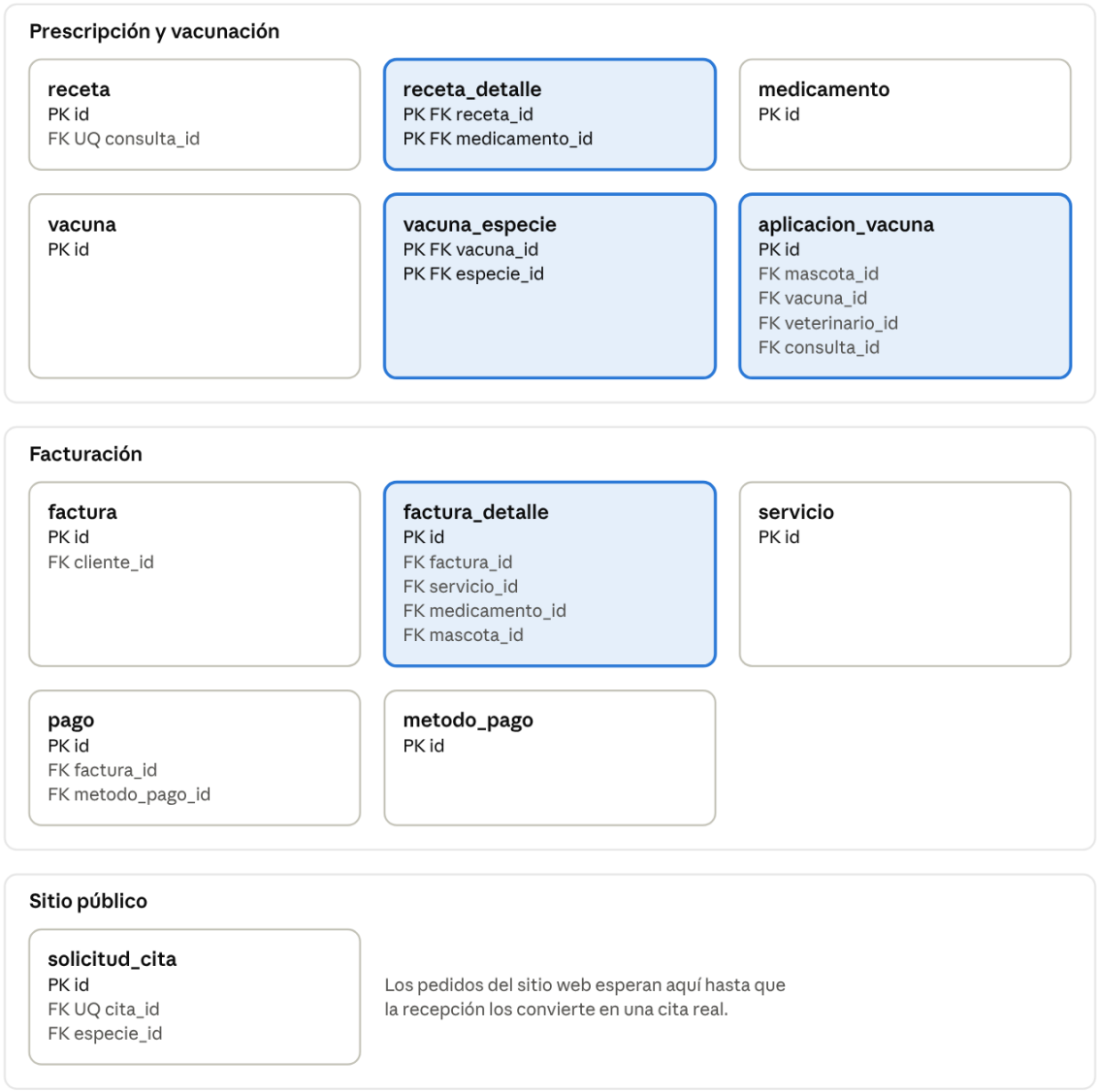



*Figura 2. Modelo relacional (2 de 2): prescripción y vacunación, facturación y sitio público.*

Las siete tablas resaltadas resuelven relaciones muchos a muchos. `mascota` es el centro del modelo: de ella dependen las citas, consultas, pesos, condiciones y vacunas.

El diagrama entidad-relación completo, con las 219 columnas, sus tipos y las líneas de cada relación con su cardinalidad, está en `modelos/diagrama_er.pdf`, en una hoja A1 que se puede ampliar sin perder nitidez.

## 4. Arquitectura del sistema

PetCare separa presentación, lógica y acceso a datos en carpetas distintas, y cada operación las recorre en el mismo orden: **formulario → acción del servidor → repositorio SQL → PostgreSQL**.




*Figura 3. Arquitectura cliente/servidor en tres capas.*

El cliente es el navegador; la aplicación corre en un servidor Next.js y los datos en un servidor PostgreSQL. Las flechas siguen el registro de un tutor, descrito paso a paso más abajo.

## Las tres capas

| Capa | Qué contiene | Archivos |
|---|---|---|
| Presentación | Formularios, menú lateral, tablas de consulta con filtros y paginación, paneles, botones y validaciones visuales | `web/src/app/(publico)/` (sitio público), `web/src/app/admin/<módulo>/page.tsx` y `Form*.tsx` (panel), `web/src/components/` (piezas compartidas) |
| Lógica | Reglas de negocio, validación de cada campo, permisos por rol, sesión y control de cada operación | `app/admin/<módulo>/acciones.ts`, `lib/validar.ts`, `lib/permisos.ts`, `lib/auth.ts`, `lib/limite.ts` |
| Física / acceso a datos | Conexión, consultas SQL, transacciones, traducción de errores y, en el servidor, tablas, vistas, funciones y triggers | `lib/db.ts`, `lib/repos/<módulo>.ts`, `lib/catalogos.ts`, `database/*.sql` |

Todas las consultas SQL están en `lib/repos/`: un archivo por módulo, más `sesion.ts` para el inicio de sesión y `reportes.ts` para los informes. Ninguna página, formulario ni acción escribe SQL. Los nueve catálogos se describen en `lib/catalogos.ts` (tabla, columnas y consulta de su lista) y los ejecuta `lib/repos/catalogos.ts`.

### Recorrido de una operación: registrar un tutor

1. **Presentación.** `FormCliente.tsx` muestra el formulario y marca los campos obligatorios. Al enviarlo llama a la acción `guardar`.
2. **Lógica.** `clientes/acciones.ts` comprueba que el usuario tenga el permiso `clientes.editar` y valida cada campo con `lib/validar.ts`: texto obligatorio, longitud máxima, formato de teléfono y de correo.
3. **Acceso a datos.** `lib/repos/clientes.ts` ejecuta `INSERT INTO cliente (...) VALUES ($1, ..., $9) RETURNING id`. Los valores viajan como parámetros, lo que evita la inyección de SQL.
4. **Base de datos.** PostgreSQL revisa dominios, `CHECK` y `UNIQUE`. Si el documento ya existe, rechaza la fila y `mensajeDeError` traduce `uq_cliente_documento` a «Ya existe un tutor con ese tipo y número de documento».
5. **Respuesta.** La acción vuelve a la ficha del tutor con el aviso «Tutor registrado».

### Conexión y seguridad

- **Pool de conexiones.** `lib/db.ts` abre un pool de hasta 10 conexiones con la librería `pg`. La contraseña de PostgreSQL no está en el código: se lee del archivo `pgpass.conf`.
- **Transacciones.** Las operaciones que tocan varias tablas usan `enTransaccion`, con `BEGIN`, `COMMIT` y `ROLLBACK`. Por ejemplo, convertir una solicitud en cita, crear una factura con su primera línea, o registrar una consulta con su peso y marcar la cita como atendida.
- **Sesión.** La cookie solo guarda el id del usuario, firmado con HMAC. El rol se lee de la base en cada petición.
- **Permisos.** Se comprueban en el servidor en cada página y en cada acción, no solo ocultando botones.
- **Contraseñas.** Se guardan con hash scrypt. Tras 5 intentos fallidos, el usuario espera 10 minutos.

## 5. Operaciones CRUD

Las cuatro operaciones se ejecutan contra PostgreSQL desde los módulos principales, no en un CRUD aislado. Ningún dato de las pantallas está escrito en el código ni vive solo en variables temporales.

| Módulo | INSERT | SELECT | UPDATE | DELETE |
|---|---|---|---|---|
| Tutores | Registrar tutor | Listado con búsqueda, ficha con mascotas, citas, consultas y facturas | Editar datos; dar de baja o reactivar | No se borra (ver nota) |
| Mascotas | Registrar mascota, peso y condición médica | Listado y ficha con historial, vacunas, peso y citas | Editar datos, cambiar de tutor, dar de baja | Quitar una condición médica |
| Citas y solicitudes | Agendar cita; convertir solicitud en cita; el sitio público inserta la solicitud | Agenda del día y del mes, detalle de cita, bandeja de solicitudes | Editar, confirmar, cancelar con motivo, marcar inasistencia; cambiar estado de la solicitud | — |

| Módulo | INSERT | SELECT | UPDATE | DELETE |
|---|---|---|---|---|
| Atención clínica | Consulta con peso, diagnósticos, tratamientos y receta con sus líneas | Listados de consultas, tratamientos y recetas; receta imprimible | Editar consulta, tipo de diagnóstico, estado del tratamiento | Quitar un diagnóstico o una línea de la receta |
| Vacunación | Aplicar vacuna | Aplicaciones y refuerzos pendientes | — | — |
| Facturación | Crear factura con su primera línea, agregar líneas, registrar pagos | Listado con saldo, detalle de factura y pagos | Descuento e impuesto, anular factura, devolver stock | Quitar una línea o un pago |
| Personal | Registrar veterinario con especialidades; crear usuario | Listados de veterinarios y usuarios | Editar veterinario; cambiar rol, estado o contraseña del usuario | Reemplazar las especialidades del veterinario |
| Catálogos | Agregar especie, raza, medicamento, vacuna, servicio, diagnóstico, condición, especialidad o método de pago | Listado con búsqueda de cada catálogo | Editar cualquier registro | Eliminar un registro que no esté en uso |

**Nota sobre DELETE.** Tutores, mascotas, consultas y facturas no se borran: el historial clínico y contable debe conservarse. Para ellos se usa una baja lógica (`UPDATE ... SET activo = false`) o la anulación. El `DELETE` se usa donde no destruye historia: líneas, pagos, diagnósticos y catálogos sin uso. Si se intenta eliminar un catálogo en uso, como la especie *Canino*, la clave foránea con `RESTRICT` lo impide y la pantalla muestra el motivo.

### Ejemplos del código

```
-- INSERT · lib/repos/clientes.ts
INSERT INTO cliente (tipo_documento, documento, nombres, apellidos, telefono, email, direccion,
ciudad, notas)
VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9) RETURNING id;

-- SELECT · lib/repos/clientes.ts (saldo del tutor desde una vista)
SELECT COALESCE(sum(f.saldo), 0) FROM v_factura_resumen f
 WHERE f.cliente_id = c.id AND f.estado <> 'anulada';

-- UPDATE · lib/repos/consultas.ts
UPDATE tratamiento SET estado = $2 WHERE id = $1;

-- DELETE · lib/repos/mascotas.ts
DELETE FROM mascota_condicion WHERE mascota_id = $1 AND condicion_medica_id = $2;
```

## 6. Módulos funcionales

El sistema tiene once módulos: el sitio público y diez del panel interno. Todos siguen el mismo camino de capas: la página del módulo, su archivo `acciones.ts` y su repositorio en `lib/repos/`. El menú lateral muestra a cada rol solo los módulos que puede usar.

## Sitio público y solicitudes de cita

- **Objetivo:** mostrar los servicios, el equipo y los datos de la clínica, y recibir pedidos de cita sin crear cuenta.
- **Actor:** visitante o tutor.
- **Tablas:** `servicio`, `veterinario`, `veterinario_especialidad`, `especialidad`, `especie` y `solicitud_cita`.
- **Uso de la base:** lee los servicios marcados como visibles y los veterinarios activos; guarda el pedido en `solicitud_cita` con estado *nueva*.
- **Operaciones:** `SELECT` e `INSERT`.
- **Relaciones:** `veterinario` → `veterinario_especialidad` → `especialidad`; `solicitud_cita` → `especie`.
- **Conexión:** la solicitud aparece en el tablero y en el módulo de citas, donde la recepción la convierte en cita.

## Tablero

- **Objetivo:** dar un resumen del día y avisar lo que necesita atención.
- **Actor:** todos los roles. El veterinario ve solo sus citas.
- **Tablas y vistas:** `mascota`, `cliente`, `cita`, `consulta`, `pago`, `factura`, `solicitud_cita`, `v_agenda`, `v_vacunas_pendientes`, `v_stock_alerta` y `v_ingresos_mensuales`.
- **Uso de la base:** cuenta mascotas, tutores, citas de hoy y consultas del mes; suma lo cobrado en el mes; avisa vacunas vencidas, stock bajo, solicitudes nuevas, facturas por cobrar y citas sin cerrar.
- **Operaciones:** `SELECT`.
- **Conexión:** cada aviso lleva al módulo donde se resuelve.

## Tutores

- **Objetivo:** registrar y mantener a los propietarios de las mascotas.
- **Actor:** recepcionista y administrador. El veterinario solo consulta.
- **Tablas y vistas:** `cliente`, `mascota`, `consulta`, `v_mascota_perfil`, `v_agenda` y `v_factura_resumen`.
- **Uso de la base:** busca por nombre, documento o teléfono; la ficha reúne sus mascotas, citas, consultas, facturas y el saldo pendiente.
- **Operaciones:** `INSERT`, `SELECT` y `UPDATE` (incluida la baja lógica).
- **Relaciones:** `cliente` 1:N `mascota`; `cliente` 1:N `factura`.
- **Conexión:** desde la ficha se registra una mascota o una factura para ese tutor.

## Mascotas

- **Objetivo:** llevar la ficha y el historial clínico completo de cada paciente.

- **Actor:** los tres roles. Cambiar de tutor y dar de baja es solo de recepción y administración.
- **Tablas y vistas:** `mascota`, `especie`, `raza`, `condicion_medica`, `mascota_condicion`, `registro_peso`, `v_mascota_perfil`, `v_historial_clinico` y `v_vacunas_pendientes`, además de las tablas clínicas para las pestañas.
- **Uso de la base:** la ficha calcula edad y peso actual con vistas, y muestra en pestañas el historial, las vacunas, la gráfica de peso, las citas y las facturas.
- **Operaciones:** `INSERT` (mascota, peso, condición), `SELECT`, `UPDATE` (datos y baja) y `DELETE` (condición médica).
- **Relaciones:** clave foránea compuesta `(raza_id, especie_id)` para que la raza sea de la especie; `mascota` N:M `condicion_medica` por `mascota_condicion`.
- **Conexión:** es el punto de partida para agendar citas, atender consultas y aplicar vacunas.

## Citas y solicitudes web

- **Objetivo:** organizar la agenda de los veterinarios y convertir los pedidos del sitio público en citas.
- **Actor:** recepcionista y administrador; el veterinario gestiona solo sus citas.
- **Tablas y vistas:** `cita`, `mascota`, `veterinario`, `servicio`, `solicitud_cita`, `especie` y `v_agenda`.
- **Uso de la base:** inserta la cita; la restricción `EXCLUDE` rechaza un cruce de horario. Al convertir una solicitud, inserta la cita y marca la solicitud como *convertida* en una sola transacción.
- **Operaciones:** `INSERT`, `SELECT` y `UPDATE` (confirmar, cancelar con motivo, inasistencia).
- **Relaciones:** `cita` → `mascota`, `veterinario` y `servicio`; `solicitud_cita` 1:1 `cita`.
- **Conexión:** una cita del día se abre en Atención clínica con el botón **Atender**.

## Atención clínica

- **Objetivo:** registrar la consulta con sus signos vitales, diagnósticos, tratamientos y receta.
- **Actor:** veterinario y administrador. La recepción no ve esta parte.
- **Tablas y vistas:** `consulta`, `cita`, `registro_peso`, `diagnostico`, `consulta_diagnostico`, `tratamiento`, `receta`, `receta_detalle`, `medicamento`, `mascota_condicion` y `v_consulta_diagnosticos`.
- **Uso de la base:** en una transacción inserta la consulta, guarda el peso y marca la cita como *atendida*. Luego agrega diagnósticos del catálogo, tratamientos y la receta, cuyo folio pone la base. Muestra las alergias de la mascota como alerta.
- **Operaciones:** `INSERT`, `SELECT`, `UPDATE` y `DELETE` (diagnóstico o línea de receta).
- **Relaciones:** `cita` 1:1 `consulta`; `consulta` 1:1 `receta`; `consulta` N:M `diagnostico`; `receta` N:M `medicamento`.
- **Conexión:** solo el veterinario que firmó un registro, o un administrador, puede modificarlo. La receta se imprime desde su detalle.

## Vacunación

- **Objetivo:** registrar las vacunas aplicadas y controlar los refuerzos.
- **Actor:** veterinario y administrador aplican; la recepción consulta los refuerzos pendientes.
- **Tablas y vistas:** `aplicacion_vacuna`, `vacuna`, `vacuna_especie`, `mascota`, `veterinario` y `v_vacunas_pendientes`.
- **Uso de la base:** solo ofrece vacunas indicadas para la especie; la aplicación revisa la edad mínima y un trigger calcula la fecha del próximo refuerzo.
- **Operaciones:** `INSERT` y `SELECT`.
- **Relaciones:** `mascota` N:M `vacuna` por `aplicacion_vacuna`; `vacuna` N:M `especie` por `vacuna_especie`.
- **Conexión:** los refuerzos vencidos aparecen como aviso en el tablero y en la ficha de la mascota.

## Facturación y pagos

- **Objetivo:** cobrar servicios y medicamentos al tutor y llevar el saldo de cada factura.
- **Actor:** recepcionista y administrador.
- **Tablas y vistas:** `factura`, `factura_detalle`, `servicio`, `medicamento`, `pago`, `metodo_pago`, `cliente`, `mascota` y `v_factura_resumen`.
- **Uso de la base:** crea la factura con su primera línea; los triggers ponen el número, recalculan el total y el estado, descuentan el stock e impiden cobrar más del total. Al quitar una línea o anular la factura, la aplicación devuelve el stock.
- **Operaciones:** `INSERT`, `SELECT`, `UPDATE` y `DELETE` (línea o pago).
- **Relaciones:** `cliente` 1:N `factura`; `factura` 1:N `factura_detalle` y `pago`; cada línea apunta a un `servicio` o a un `medicamento`.
- **Conexión:** el saldo aparece en la ficha del tutor y los ingresos alimentan los reportes.

## Personal y usuarios

- **Objetivo:** administrar a los veterinarios y las cuentas de acceso.
- **Actor:** administrador.
- **Tablas:** `veterinario`, `veterinario_especialidad`, `especialidad`, `usuario` y `rol`.
- **Uso de la base:** registra al veterinario con sus especialidades y una principal; crea usuarios con su rol y los vincula a un veterinario para que pueda firmar.
- **Operaciones:** `INSERT`, `SELECT`, `UPDATE` y `DELETE` (reemplazo de especialidades).
- **Relaciones:** `usuario` 1:1 `veterinario`; `rol` 1:N `usuario`; `veterinario` N:M `especialidad`.
- **Conexión:** el rol del usuario decide qué módulos ve en el menú y qué acciones puede hacer.

## Catálogos

- **Objetivo:** mantener los datos de referencia que usan los demás módulos.

- **Actor:** administrador.
- **Tablas:** `especie`, `raza`, `medicamento`, `vacuna` (con `vacuna_especie`), `servicio`, `diagnostico`, `condicion_medica`, `especialidad` y `metodo_pago`.
- **Uso de la base:** una sola pantalla genérica lista, crea, edita y elimina los nueve catálogos.
- **Operaciones:** `INSERT`, `SELECT`, `UPDATE` y `DELETE`.
- **Relaciones:** `raza` → `especie`; un registro en uso no se puede eliminar por `RESTRICT`.
- **Conexión:** sus valores alimentan las listas desplegables de todos los formularios.

### Reportes

- **Objetivo:** convertir los datos en información para la administración.
- **Actor:** administrador.
- **Vistas:** `v_ingresos_mensuales`, `v_servicios_mas_usados` y `v_morbilidad`.
- **Operaciones:** `SELECT` con filtros por período.
- **Conexión:** se detalla en la sección 7.

## 7. Informes

Los informes no muestran tablas completas: agrupan, filtran y calculan sobre varias tablas a la vez. Tres están en el módulo *Reportes* y los demás alimentan el tablero y las pantallas de vacunación y facturación. Cada uno lee una vista de la base, de modo que el cálculo vive en un solo lugar.

| Informe | Pregunta que responde | Tablas que une | Cálculo en SQL |
|---|---|---|---|
| Ingresos mensuales (`v_ingresos_mensuales`) | ¿Cuánto se facturó, cuánto se cobró y cuánto falta por cobrar cada mes? | `factura`, `pago` | `date_trunc('month')`, `GROUP BY` mes, suma de pagos con `LEFT JOIN LATERAL`, sin facturas anuladas. Filtro *desde* y *hasta* por mes |
| Servicios más usados (`v_servicios_mas_usados`) | ¿Qué servicios se venden más y cuánto dejan? | `factura_detalle`, `servicio`, `factura` | `count(*)`, `sum(cantidad)`, `sum(subtotal)`, `GROUP BY` servicio, los 10 primeros |
| Diagnósticos más frecuentes (`v_morbilidad`) | ¿Qué enfermedades se atienden más y en cuántos pacientes? | `consulta_diagnostico`, `diagnostico`, `consulta` | `count(*) FILTER (WHERE tipo = 'DEFINITIVO')`, `count(DISTINCT mascota_id)`, sin descartados |
| Refuerzos de vacuna (`v_vacunas_pendientes`) | ¿Qué mascotas tienen la vacuna vencida o por vencer en 30 días, y a quién llamar? | `aplicacion_vacuna`, `mascota`, `especie`, `cliente`, `vacuna` | `CASE` contra la fecha de hoy y subconsulta con `max(fecha_aplicacion)` para tomar solo la última dosis |
| Medicamentos por agotarse (`v_stock_alerta`) | ¿Qué medicamentos están en su stock mínimo o sin existencias? | `medicamento` | `stock <= stock_minimo` y `CASE` entre *sin stock* y *bajo* |
| Saldo de facturas (`v_factura_resumen`) | ¿Cuánto ha pagado cada tutor y cuánto debe? | `factura`, `cliente`, `pago`, `factura_detalle` | Total menos la suma de pagos, calculado y nunca guardado |

| Informe | Pregunta que responde | Tablas que une | Cálculo en SQL |
|---|---|---|---|
| Indicadores del tablero | ¿Cómo va el día y el mes? | `cita`, `consulta`, `mascota`, `cliente`, `pago`, `especie` | Conteos de citas de hoy y consultas del mes, cobrado del mes y mascotas por especie con `GROUP BY` |

El módulo *Reportes* muestra arriba tres cifras del período elegido (facturado, cobrado y por cobrar) y debajo la tabla mensual y dos gráficas de barras con los servicios y los diagnósticos. El reporte de morbilidad solo es posible porque el diagnóstico es un catálogo y no texto libre.

```
-- v_morbilidad · database/05_vistas.sql
SELECT d.id, d.codigo, d.nombre, d.categoria,
       count(*) FILTER (WHERE cd.tipo = 'DEFINITIVO') AS confirmados,
       count(*)                                       AS casos,
       count(DISTINCT c.mascota_id)                   AS pacientes
  FROM consulta_diagnostico cd
  JOIN diagnostico d ON d.id = cd.diagnostico_id
  JOIN consulta c    ON c.id = cd.consulta_id
 WHERE cd.tipo <> 'DESCARTADO'
 GROUP BY d.id, d.codigo, d.nombre, d.categoria;
```

## 8. Entregables y estructura del comprimido

Los diez entregables que pide el parcial van en un único archivo comprimido, organizado como se muestra a continuación.

| Entregable del parcial | Dónde queda en el comprimido |
|---|---|
| Código fuente completo | `codigo_fuente/PetCare/` |
| Backup de la base de datos | `base_datos/petcare.backup` |
| Script SQL del esquema | `base_datos/esquema.sql` y `base_datos/scripts/` |
| Modelo relacional | `modelos/modelo_relacional.pdf` |
| Diagrama con mínimo 12 tablas | `modelos/diagrama_er.pdf` |
| Descripción del problema | `documentacion/descripcion_proyecto.pdf` |
| Descripción de los módulos | Este documento, sección 6 |
| Explicación de las tres capas | Este documento, sección 4 |
| Enlace del video | `enlace_video.txt` y sección 9 |
| Instrucciones de ejecución | Este documento, sección 10, y `README.md` |

### Estructura del archivo comprimido

```
SEGUNDO_PARCIAL_PETCARE/
│
├── codigo_fuente/
│   └── PetCare/            web/, database/, empaquetado/ y README.md
│
├── base_datos/
│   ├── esquema.sql         estructura completa en un solo archivo
│   ├── petcare.backup      backup con datos (pg_dump, formato custom)
│   └── scripts/            01_esquema.sql … 08_pruebas.sql y petcare.sql
│
├── modelos/
│   ├── modelo_relacional.pdf
│   └── diagrama_er.pdf     diagrama entidad-relación completo (hoja A1)
│
├── documentacion/
│   └── descripcion_proyecto.pdf
│
└── enlace_video.txt
```

### Generar el esquema y el backup

Los dos archivos se generaron el 4 de noviembre de 2026 en `entrega/base_datos/` y se probaron en bases vacías. Ambos recrean las 29 tablas, 10 vistas, 39 claves foráneas, 56 `CHECK`, 10 triggers y 104 índices; el backup trae además los mismos datos en las 29 tablas, y las 41 pruebas de `08_pruebas.sql` pasaron sobre la copia restaurada.

Se ejecutan en una terminal, desde la carpeta `base_datos/`. Si `pg_dump` no se reconoce, está en `C:\Program Files\PostgreSQL\16\bin`.

```
pg_dump -U postgres -d petcare --schema-only --no-owner -f esquema.sql

pg_dump -U postgres -d petcare -Fc --no-owner -f petcare.backup
```

Para restaurar el backup en otro equipo se crea una base vacía y se usa `pg_restore`:

```
createdb -U postgres -E UTF8 -T template0 petcare

pg_restore -U postgres -d petcare --no-owner petcare.backup
```

## 9. Video explicativo en YouTube

**Enlace:** https://youtu.be/KULm3711eSk (público, 1:53).

El video presenta PetCare como propuesta para una clínica veterinaria y muestra el sistema funcionando sobre la base de datos.

| Parte | Qué se ve |
|---|---|
| Problema | Cómo trabaja una clínica sin sistema: citas en cuadernos, historial en carpetas y cuentas en Excel |
| Solicitud y cita | Un cliente pide cita desde `/agendar`; la recepción la convierte en cita, y la restricción `EXCLUDE` rechaza un cruce de horario |
| Ficha de la mascota | Consultas, vacunas, evolución del peso y próximas citas; al registrar una vacuna solo aparecen las de su especie |

| Parte | Qué se ve |
|---|---|
| Atención clínica | Consulta, diagnóstico, tratamiento y receta imprimible; el `CHECK` rechaza una temperatura de 60 °C |
| Facturación | Al facturar un medicamento el stock baja solo (trigger); un pago parcial cambia el estado de la factura |
| Reportes | Ingresos y enfermedades más frecuentes |
| Permisos | La recepción no entra a la parte clínica y cada veterinario solo modifica lo que firmó |

# 10. Instrucciones para ejecutar el proyecto

PetCare se ejecuta en un computador con Node.js 20 o superior y PostgreSQL 16, y se abre en el navegador en `http://localhost:3000`.

## 1. Crear la base de datos

Hay dos caminos. El primero restaura el backup con los datos de demostración:

```
createdb -U postgres -E UTF8 -T template0 petcare

pg_restore -U postgres -d petcare --no-owner base_datos/petcare.backup
```

El segundo crea la base desde los scripts, con los catálogos y los datos de demostración:

```
psql -U postgres -c "CREATE DATABASE petcare WITH ENCODING 'UTF8' TEMPLATE template0"

psql -U postgres -d petcare -f base_datos/scripts/petcare.sql
```

Para comprobar la base se pueden correr las 41 pruebas de `08_pruebas.sql`. Se ejecutan en una transacción que se revierte, así que no dejan datos.

## 2. Configurar la aplicación

1. En la carpeta `web`, copiar `.env.example` con el nombre `.env.local`.
2. Cambiar `AUTH_SECRET` por un texto largo y aleatorio.
3. Escribir la contraseña de PostgreSQL en la línea `PGPASSWORD`, quitándole el `#`. También puede ir en el archivo `pgpass.conf` de PostgreSQL.

## 3. Instalar y ejecutar

La primera vez, dentro de la carpeta `web`, se instalan las dependencias:

```
npm install
```

Luego se compila y se inicia la versión de producción. La compilación necesita internet la primera vez, porque descarga las fuentes de la página.

```
npm run build

npm start
```

Para desarrollo basta con `npm run dev`. En ambos casos el sistema queda en `http://localhost:3000`.

## 4. Usuarios de prueba

La contraseña de todos es `petcare2026` y solo sirve para la demostración. El sitio público no pide usuario; el panel se abre desde **Acceso**.

| Usuario | Rol | Qué puede hacer |
|---|---|---|
| `vsandoval` | Administrador | Todo el sistema |
| `lferreyra`, `mocampo`, `sbrenner` | Veterinario | Leer el historial de cualquier mascota y modificar solo sus propios registros y citas |
| `cruiz` | Recepcionista | Agenda, solicitudes web, tutores, mascotas y facturación |

# Anexo. Modelo relacional completo

Este anexo se generó el 4 de noviembre de 2026 leyendo el catálogo de PostgreSQL de la base `petcare`, así que coincide columna por columna con lo implementado. Lista las 29 tablas con sus 219 columnas, tipos y claves.

**Cómo leerlo:** la clave primaria va en **negrita** con `PK`; `FK→tabla` indica la tabla a la que apunta la clave foránea; `(nulo)` marca las columnas opcionales, y todas las demás son `NOT NULL`. `dom_dinero`, `dom_email` y `dom_telefono` son los dominios de la sección 3. En `mascota`, la pareja `(raza_id, especie_id)` es además una clave foránea compuesta hacia `raza (id, especie_id)`.

## Seguridad y personas

- **rol** (**id** smallint PK, nombre varchar(50), descripcion varchar(200) (nulo))
- **usuario** (**id** int PK, rol_id smallint FK→rol, nombre_usuario varchar(50), email dom_email, password_hash varchar(255), activo boolean, ultimo_acceso timestamptz (nulo), creado_en timestamptz)
- **especialidad** (**id** smallint PK, nombre varchar(80), descripcion varchar(200) (nulo))
- **veterinario** (**id** int PK, usuario_id int FK→usuario (nulo), documento varchar(20), matricula varchar(30), nombres varchar(80), apellidos varchar(80), telefono dom_telefono (nulo), email dom_email (nulo), foto_url varchar(300) (nulo), biografia text (nulo), activo boolean)
- **veterinario_especialidad** (**veterinario_id** int PK FK→veterinario, **especialidad_id** smallint PK FK→especialidad, es_principal boolean)
- **cliente** (**id** int PK, tipo_documento varchar(10), documento varchar(20), nombres varchar(80), apellidos varchar(80), telefono dom_telefono, email dom_email (nulo), direccion varchar(200) (nulo), ciudad varchar(80) (nulo), notas text (nulo), activo boolean, creado_en timestamptz)

## Mascota

- **especie** (**id** smallint PK, nombre varchar(50), icono varchar(16) (nulo), esperanza_vida_anios smallint (nulo))
- **raza** (**id** int PK, especie_id smallint FK→especie, nombre varchar(80), tamanio varchar(15) (nulo))
- **mascota** (**id** int PK, cliente_id int FK→cliente, especie_id smallint FK→especie, raza_id int FK→raza (nulo), nombre varchar(60), sexo char(1), fecha_nacimiento date (nulo), color varchar(60) (nulo), microchip varchar(25) (nulo), esterilizado boolean, foto_url varchar(300) (nulo), observaciones text (nulo), fecha_fallecimiento date (nulo), activo boolean, creado_en timestamptz)
- **condicion_medica** (**id** int PK, nombre varchar(100), tipo varchar(15), descripcion varchar(300) (nulo))
- **mascota_condicion** (**mascota_id** int PK FK→mascota, **condicion_medica_id** int PK FK→condicion_medica, severidad varchar(10), fecha_deteccion date (nulo), notas text (nulo))
- **registro_peso** (**id** int PK, mascota_id int FK→mascota, consulta_id int FK→consulta (nulo), fecha_registro date, peso_kg numeric(6,2), observacion varchar(200) (nulo))

## Atención clínica

- **cita** (**id** int PK, mascota_id int FK→mascota, veterinario_id int FK→veterinario, servicio_id int FK→servicio (nulo), fecha_hora_inicio timestamptz, duracion_minutos smallint, fecha_hora_fin timestamptz, motivo varchar(300), estado varchar(12), motivo_cancelacion varchar(250) (nulo), notas text (nulo), creado_en timestamptz)

- **consulta** (**id** int PK, mascota_id int FK→mascota, veterinario_id int FK→veterinario, cita_id int FK→cita (nulo), fecha_hora timestamptz, tipo varchar(15), motivo varchar(300), anamnesis text (nulo), temperatura_c numeric(4,1) (nulo), frecuencia_cardiaca smallint (nulo), frecuencia_respiratoria smallint (nulo), examen_fisico text (nulo), observaciones text (nulo), proxima_revision date (nulo), creado_en timestamptz)
- **diagnostico** (**id** int PK, codigo varchar(15), nombre varchar(120), categoria varchar(50) (nulo))
- **consulta_diagnostico** (**consulta_id** int PK FK→consulta, **diagnostico_id** int PK FK→diagnostico, tipo varchar(12), es_principal boolean, notas text (nulo))
- **tratamiento** (**id** int PK, consulta_id int FK→consulta, descripcion varchar(300), indicaciones text (nulo), fecha_inicio date, fecha_fin_estimada date (nulo), estado varchar(12))

## Prescripción y vacunación

- **receta** (**id** int PK, consulta_id int FK→consulta, folio varchar(20), fecha_emision date, indicaciones_generales text (nulo), vigencia_dias smallint)
- **medicamento** (**id** int PK, nombre varchar(120), principio_activo varchar(120), presentacion varchar(20), concentracion varchar(40) (nulo), laboratorio varchar(100) (nulo), requiere_receta boolean, stock int, stock_minimo int, precio dom_dinero, activo boolean)
- **receta_detalle** (**receta_id** int PK FK→receta, **medicamento_id** int PK FK→medicamento, dosis varchar(60), frecuencia varchar(60), duracion_dias smallint, via_administracion varchar(15), cantidad numeric(8,2), instrucciones varchar(300) (nulo))
- **vacuna** (**id** int PK, nombre varchar(100), laboratorio varchar(100) (nulo), enfermedades_previene varchar(300) (nulo), intervalo_refuerzo_meses smallint (nulo), edad_minima_semanas smallint (nulo), precio dom_dinero, activo boolean)
- **vacuna_especie** (**vacuna_id** int PK FK→vacuna, **especie_id** smallint PK FK→especie)
- **aplicacion_vacuna** (**id** int PK, mascota_id int FK→mascota, vacuna_id int FK→vacuna, veterinario_id int FK→veterinario, consulta_id int FK→consulta (nulo), fecha_aplicacion date, fecha_proximo_refuerzo date (nulo), lote varchar(40), dosis_numero smallint, observaciones text (nulo))

## Facturación

- **servicio** (**id** int PK, codigo varchar(15), nombre varchar(120), categoria varchar(30), descripcion text (nulo), precio_base dom_dinero, duracion_estimada_min smallint (nulo), icono varchar(16) (nulo), visible_publico boolean, activo boolean)
- **factura** (**id** int PK, cliente_id int FK→cliente, numero varchar(20), fecha_emision date, subtotal dom_dinero, descuento dom_dinero, impuesto dom_dinero, total dom_dinero, estado varchar(12), observaciones text (nulo), creado_en timestamptz)
- **factura_detalle** (**id** int PK, factura_id int FK→factura, servicio_id int FK→servicio (nulo), medicamento_id int FK→medicamento (nulo), mascota_id int FK→mascota (nulo), linea_nro smallint, descripcion varchar(200), cantidad numeric(8,2), precio_unitario dom_dinero, subtotal numeric(12,2) generada)
- **metodo_pago** (**id** smallint PK, nombre varchar(40), requiere_referencia boolean, activo boolean)
- **pago** (**id** int PK, factura_id int FK→factura, metodo_pago_id smallint FK→metodo_pago, fecha_hora timestamptz, monto dom_dinero, referencia varchar(60) (nulo))

## Sitio público

- **solicitud_cita** (**id** int PK, cita_id int FK→cita (nulo), especie_id smallint FK→especie (nulo), nombre_contacto varchar(120), telefono_contacto dom_telefono, email_contacto varchar(120) (nulo), nombre_mascota varchar(60) (nulo), motivo varchar(300), fecha_preferida date (nulo), franja_preferida varchar(12) (nulo), estado varchar(12), creado_en timestamptz)